Azure Question Index - All 21 Questions

Q#	Question
Q1	Azure global infrastructure - regions, AZs, resource groups, subscriptions
Q2	Azure AD / Entra ID - identity, managed identities, service principals, RBAC
Q3	Azure VMs - sizes, availability sets, scale sets, Spot VMs
Q4	Azure Storage - Blob tiers, Files, Queues, Tables, Data Lake Gen2
Q5	Azure VNet - NSGs, Azure Firewall, peering, Private Endpoints, service endpoints
Q6	Azure SQL Database - tiers, elastic pools, geo-replication
Q7	AKS - node pools, virtual nodes, Azure CNI, workload identity
Q8	Azure Functions - triggers, bindings, durable functions, premium plan
Q9	Azure App Service - PaaS web hosting, deployment slots, scaling
Q10	Cosmos DB - multi-model, global distribution, consistency levels
Q11	Azure Monitor + Log Analytics + Application Insights
Q12	Azure Key Vault - secrets, keys, certificates, access policies vs RBAC
Q13	Azure DevOps - Pipelines, Repos, Boards, Artifacts
Q14	Azure Policy + Blueprints + Management Groups - governance
Q15	ARM templates vs Bicep vs Terraform on Azure
Q16	Azure Front Door + CDN - global traffic management
Q17	Azure Container Apps - serverless containers, Dapr, KEDA
Q18	AKS cluster out of IP addresses - Azure CNI exhaustion
Q19	Azure managed identity not working - RBAC debugging
Q20	Azure Storage account accidentally exposed publicly
Q21	Azure Function cold start causing SLA breach

SECTION 1 - Junior Core Azure (Q1-Q6)

Q1 Azure global infrastructure - regions, AZs, resource groups, subscriptions

The Simple Answer

Azure organizes resources in a hierarchy: Tenant (Azure AD) → Management Groups → Subscriptions → Resource Groups → Resources. Azure has 60+ regions globally, many with 3 Availability Zones. Resource Groups are logical containers that group related resources - they are the unit of deployment, management, and billing attribution.

Key Differences from AWS

Resource Groups are required in Azure - every resource must belong to one. AWS has optional resource groups. Azure Resource Groups can span regions (an RG can contain resources in US East and Europe West). This is useful for grouping an entire application's resources together regardless of location.

Subscriptions are the billing and access boundary, similar to AWS accounts. An organization typically has multiple subscriptions - one for production, one for development, one for shared infrastructure.

Management Groups provide hierarchy above subscriptions for applying policies and RBAC at scale. Up to 6 levels of nesting. Equivalent to AWS Organizations OUs.

Availability Zones

Azure AZs work the same as AWS - physically separate data centers within a region. Not all Azure regions have AZs. For HA, deploy across AZs with zone-redundant services. Azure additionally offers Availability Sets - logical grouping of VMs across fault domains and update domains within a single data center, providing HA without AZs.

> **Azure naming quirk:** Azure region names are verbose (East US, West Europe, Southeast Asia) while AWS uses codes (us-east-1, eu-west-1). Azure also has paired regions - each region has a partner for DR, and some services (like Geo-Redundant Storage) automatically replicate to the paired region.

Q2 Azure AD / Entra ID - identity, managed identities, service principals, RBAC

The Simple Answer

Azure AD (now Microsoft Entra ID) is the identity foundation of Azure. It handles authentication for users, applications, and services. Every Azure subscription is associated with an Azure AD tenant.

Key Concepts

Users and Groups - Human identities. Groups simplify RBAC by assigning roles to groups instead of individual users.

Service Principals - Application identities. Created when you register an app in Azure AD. Used for automation, CI/CD pipelines, and service-to-service auth. Equivalent to AWS IAM roles for applications.

Managed Identities - Azure-managed service principals for Azure resources. No credential management needed - Azure handles the identity lifecycle automatically.

System-assigned - Tied to a specific resource (VM, Function, AKS). Deleted when the resource is deleted.

User-assigned - Independent lifecycle. Can be shared across multiple resources. Created and managed by you.

RBAC

Azure RBAC assigns roles at a scope: Management Group → Subscription → Resource Group → Resource. Roles are inherited downward. Built-in roles: Owner (full access), Contributor (manage but not grant access), Reader (view only). Custom roles for fine-grained control.

Managed identities are the #1 security best practice in Azure. Never store credentials in code, environment variables, or Key Vault when a managed identity can be used instead. The identity is automatic, rotated by Azure, and eliminates credential management entirely. Equivalent to AWS IAM roles for EC2/Lambda.

Q3 Azure VMs - sizes, availability sets, scale sets, Spot

The Simple Answer

Azure VMs come in families similar to AWS EC2: General purpose (D, B series), Compute optimized (F series), Memory optimized (E, M series), Storage optimized (L series), GPU (N series).

B-Series (Burstable)

Like AWS T instances - baseline CPU with credits for bursting. Cost-effective for workloads with variable CPU usage (development, small web servers).

Availability Sets

Distribute VMs across fault domains (separate racks) and update domains (staggered patching). Provides 99.95% SLA. Used when AZs are not available in the region or when you need anti-affinity within a data center.

Virtual Machine Scale Sets (VMSS)

Equivalent to AWS Auto Scaling Groups. Manage a fleet of identical VMs. Autoscale based on metrics. Rolling upgrades. Integration with Azure Load Balancer and Application Gateway.

Spot VMs

Up to 90% discount. Azure can evict with 30-second notice. Best for batch processing, CI/CD, dev/test. Configure max price - eviction when Azure spot price exceeds your max.

Q4 Azure Storage - Blob, Files, Queues, Tables, Data Lake Gen2

The Simple Answer

Azure Storage accounts contain multiple storage services under one account:

Blob Storage - Object storage equivalent to AWS S3. Tiers: Hot (frequently accessed), Cool (infrequent, 30-day minimum), Cold (rare, 90-day minimum), Archive (offline, hours to retrieve, 180-day minimum). Lifecycle management moves blobs between tiers automatically.

Azure Files - Managed file shares accessible via SMB and NFS protocols. Mount from VMs, containers, and on-premises. Equivalent to AWS EFS but supports SMB (Windows-friendly).

Queue Storage - Simple message queue. Up to 64 KB per message. Equivalent to AWS SQS Standard but fewer features. For advanced messaging, use Azure Service Bus.

Table Storage - NoSQL key-value store. Simple, cheap, low-scale. For serious NoSQL, use Cosmos DB.

Data Lake Storage Gen2 - Blob Storage with hierarchical namespace enabled. Optimized for big data analytics. Integrates with Spark, Databricks, and Synapse Analytics. Equivalent to AWS S3 + Lake Formation.

Access Tiers and Cost

Hot → Cool → Cold → Archive mirrors S3 Standard → IA → Glacier. Set lifecycle policies to automatically tier data by age. Without policies, storage costs grow linearly.

Q5 Azure VNet - NSGs, Azure Firewall, peering, Private Endpoints

The Simple Answer

Azure VNets are regional (like AWS VPCs). Subnets within a VNet can span AZs (unlike AWS where subnets are AZ-specific). NSGs (Network Security Groups) control traffic at the subnet or NIC level.

NSGs

Stateful security rules (like AWS Security Groups). Applied at subnet or individual NIC level. Default rules allow intra-VNet traffic and outbound internet. Rules have priority numbers (100-4096, lower = higher priority). Both allow and deny rules supported (unlike AWS SGs which are allow-only).

Azure Firewall

Centralized network security service. Stateful firewall with built-in HA and unlimited scalability. Application rules (FQDN-based), network rules (IP-based), NAT rules. Integrates with Azure Monitor for logging. More capable than NSGs - supports FQDN filtering, threat intelligence, TLS inspection.

VNet Peering

Connect two VNets for private communication. Global peering works cross-region. Non-transitive (same as AWS). Traffic stays on Microsoft backbone.

Private Endpoints

Create a private IP in your VNet for an Azure PaaS service (Storage, SQL, Key Vault). Traffic goes over the Microsoft backbone, never the internet. Equivalent to AWS PrivateLink Interface Endpoints. Cost: ~$7.30/month per endpoint.

Service Endpoints

Enable optimized routing from a subnet to Azure services. Traffic stays on Azure backbone. Simpler than Private Endpoints but less secure (the service still has a public endpoint - it just gets a firewall rule allowing your VNet). Free.

> **Private Endpoints vs Service Endpoints:** Private Endpoints give the service a private IP in your VNet (stronger isolation, works from on-premises via VPN/ExpressRoute). Service Endpoints just optimize routing (simpler but less isolated). For production, use Private Endpoints.

Q6 Azure SQL Database - tiers, elastic pools, geo-replication

The Simple Answer

Azure SQL Database is a fully managed relational database based on SQL Server. Multiple purchase models: DTU-based (bundled CPU, I/O, memory) and vCore-based (independently configurable). Equivalent to AWS RDS SQL Server but with Azure-specific features.

Service Tiers

General Purpose - Standard workloads. Remote storage. Equivalent to AWS RDS standard. **Business Critical** - High IOPS with local SSD. Built-in read replicas. Equivalent to RDS Multi-AZ with read replica. **Hyperscale** - Up to 100 TB. Rapid scale-up. Near-instant backups. Unique to Azure.

Elastic Pools

Share resources across multiple databases. One pool with 200 DTUs can serve 50 databases that each peak at different times. Cost-effective for SaaS applications with many tenant databases. No AWS equivalent - in AWS you would use Aurora Serverless or separate RDS instances.

Geo-Replication

Active geo-replication creates readable secondary databases in other regions. Automatic failover groups provide transparent DNS endpoint that switches to the secondary on failure. RPO: 5 seconds. RTO: 30 seconds.

SECTION 2 - Mid-Level Azure Deep Dives (Q7-Q17)

Q7 AKS - node pools, virtual nodes, Azure CNI, workload identity

The Simple Answer

AKS is Azure's managed Kubernetes. Free control plane ($0/month vs $73 for EKS). You pay only for the worker nodes.

Node Pools

System node pool - Runs Kubernetes system components (CoreDNS, metrics-server). Taint with CriticalAddonsOnly to prevent application Pods from scheduling here. **User node pools** - Run application workloads. Multiple pools with different VM sizes for different workload types (CPU-heavy, memory-heavy, GPU).

Azure CNI vs Kubenet

Azure CNI - Each Pod gets a VNet IP address. Pods are directly addressable from the VNet. Security rules apply at Pod level. Limitation: IP address exhaustion on large clusters (each node reserves IPs for its maximum Pod count). **Azure CNI Overlay** - New option. Pods get IPs from a separate CIDR, not the VNet. Solves IP exhaustion while keeping Azure CNI features. Recommended for new clusters.

Kubenet - Pods get IPs from a private range. NAT required for VNet communication. Fewer IPs consumed but less integration. Not recommended for production.

Workload Identity

Federate Kubernetes ServiceAccounts with Azure AD managed identities. Pods authenticate to Azure services using federated tokens - no secrets stored. Replaced the older AAD Pod Identity. Equivalent to AWS IRSA. The recommended approach for all AKS workloads accessing Azure services.

Virtual Nodes

Serverless Pods backed by Azure Container Instances (ACI). Scale burst workloads without adding nodes. Pay per second of execution. Good for batch processing and handling traffic spikes. Equivalent to AWS EKS Fargate.

Q8 Azure Functions - triggers, bindings, durable functions, premium plan

The Simple Answer

Azure Functions is serverless compute equivalent to AWS Lambda. Code runs in response to events. Supports C#, JavaScript, Python, Java, PowerShell, Go.

Triggers and Bindings

Triggers start function execution: HTTP, Timer, Blob Storage, Queue Storage, Service Bus, Event Hub, Cosmos DB, Event Grid. **Input bindings** read data without SDK boilerplate. **Output bindings** write data without SDK code. You declare bindings in function.json - the runtime handles connections. This is unique to Azure Functions - AWS Lambda has no equivalent declarative binding system.

Durable Functions

Extension for stateful orchestration. Patterns: function chaining (A → B → C), fan-out/fan-in (parallel execution with aggregation), human interaction (wait for approval), and monitoring (periodic checks). Equivalent to AWS Step Functions but written in code (C#, JavaScript, Python) rather than JSON state machines.

Hosting Plans

Consumption - Scale to zero, pay per execution. Cold starts (1-5 seconds). 5-minute timeout (default), 10-minute max. **Premium** - Pre-warmed instances, no cold starts. VNet integration. Unlimited execution time. Higher cost. **Dedicated (App Service Plan)** - Run on App Service infrastructure. Predictable pricing. Full control.

Cold start mitigation: Premium plan eliminates cold starts with pre-warmed instances. Alternatively: keep functions warm with timer triggers, minimize package size, avoid heavy imports at the module level.

Q9 Azure App Service - PaaS web hosting, deployment slots, scaling

The Simple Answer

App Service is Azure's PaaS for hosting web applications, REST APIs, and mobile backends. Deploy code or containers. Supports .NET, Java, Node.js, Python, PHP, Ruby. No infrastructure management. Equivalent to AWS Elastic Beanstalk but more widely used and better integrated.

Deployment Slots

Create staging, testing, or canary slots alongside the production slot. Each slot has its own URL. Deploy to a staging slot, test it, then swap it with production. The swap is instant - it just changes the routing. If the new version has issues, swap back immediately. This is Azure App Service's killer feature - built-in blue-green deployment with zero-downtime swaps.

Scaling

Scale up - Increase the App Service Plan tier (more CPU, memory). **Scale out** - Add more instances (horizontal). Auto-scale based on metrics (CPU, memory, HTTP queue length, custom). Up to 30 instances on Premium, 100 on Isolated.

App Service vs Azure Functions vs Container Apps

App Service - Traditional web apps with persistent connections, WebSockets, long-running requests. **Functions** - Event-driven, short-lived, scale to zero. **Container Apps** - Containerized microservices with Dapr integration and KEDA scaling. Choose App Service for standard web applications, Functions for event glue code, Container Apps for microservices.

Q10 Cosmos DB - multi-model, global distribution, consistency levels

The Simple Answer

Cosmos DB is Azure's globally distributed, multi-model database. Supports document (MongoDB API), key-value (Table API), graph (Gremlin API), column-family (Cassandra API), and SQL (Core SQL API). Turnkey global distribution - replicate data across any Azure regions with single-digit millisecond reads globally.

Consistency Levels (Azure-Unique Feature)

Five levels from strongest to weakest: **Strong** - Reads always return the most recent committed write. Global cost: highest latency. **Bounded Staleness** - Reads may lag by a defined time or number of operations. **Session** - Within a session, reads see the session's writes. Default. Most practical. **Consistent Prefix** - Reads never see out-of-order writes. **Eventual** - Cheapest, fastest. No ordering guarantees.

This five-level consistency model is unique to Cosmos DB. AWS DynamoDB offers only eventual and strong consistency. This flexibility allows you to tune consistency per-request, balancing performance and correctness for each use case.

Pricing

Cosmos DB uses Request Units (RUs). Each operation costs RUs based on complexity. Provisioned (set RU/s) or Serverless (pay per request). Provisioned can be expensive - careful capacity planning is essential.

Q11 Azure Monitor + Log Analytics + Application Insights

The Simple Answer

Azure Monitor - The umbrella platform for all monitoring data. Collects metrics and logs from all Azure resources. Equivalent to CloudWatch.

Log Analytics - Centralized log store with KQL (Kusto Query Language) for querying. Equivalent to CloudWatch Logs Insights but more powerful. KQL supports joins, time-series analysis, machine learning operators, and rendering charts.

Application Insights - APM (Application Performance Management) for web applications. Distributed tracing, dependency mapping, exception tracking, availability testing. Equivalent to AWS X-Ray + CloudWatch Synthetics combined. Auto-instrumentation available for .NET, Java, Node.js, Python.

Alerting

Alert rules based on metrics, logs, or activity logs. Action groups define who gets notified and how (email, SMS, webhook, Logic App, Azure Function, ITSM). Smart detection uses ML to identify anomalies (sudden failure rate increase, performance degradation).

Q12 Azure Key Vault - secrets, keys, certificates

The Simple Answer

Key Vault stores and manages secrets (connection strings, passwords), keys (encryption keys with HSM backing), and certificates (TLS certificates with auto-renewal). Equivalent to AWS Secrets Manager + KMS combined.

Access Models

Vault access policy - Legacy. Per-vault permissions for users and applications. **Azure RBAC** - Recommended. Standard RBAC roles at the data plane level. More granular and consistent with other Azure RBAC.

Integration

App Service and Azure Functions can reference Key Vault secrets directly as app settings - the value is resolved at runtime. AKS uses the Secrets Store CSI Driver to mount Key Vault secrets as volumes. Managed identities authenticate to Key Vault without credentials.

> **Key Vault vs AWS Secrets Manager:** Key Vault combines secrets AND encryption keys in one service. AWS splits them into Secrets Manager and KMS. Key Vault's certificate management (including auto-renewal with integrated CAs) is more comprehensive than ACM for non-AWS-service use cases.

Q13 Azure DevOps - Pipelines, Repos, Boards, Artifacts

The Simple Answer

Azure DevOps is a complete ALM (Application Lifecycle Management) platform - source control, CI/CD, project management, and package management in one product.

Azure Repos - Git repositories hosted in Azure DevOps. Equivalent to GitHub/GitLab. **Azure Pipelines** - CI/CD with YAML pipelines. Multi-stage, multi-environment. Supports any language, any cloud. Free tier includes 1,800 minutes/month. Equivalent to GitHub Actions. **Azure Boards** - Work item tracking, Kanban boards, sprint planning. Equivalent to Jira. **Azure Artifacts** - Package feeds for npm, NuGet, Maven, pip, universal packages. Equivalent to GitHub Packages + JFrog Artifactory. **Azure Test Plans** - Manual and exploratory testing tools.

Azure DevOps vs GitHub Actions

Azure DevOps has better enterprise features - audit logging, granular permissions, integration with Azure AD, and mature release management. GitHub Actions has better developer experience, larger marketplace, and deeper GitHub integration. Many organizations use both - GitHub for code and CI, Azure DevOps for release management and work tracking.

Q14 Azure Policy + Blueprints + Management Groups - governance

The Simple Answer

Azure Policy - Evaluate and enforce rules on resources. Examples: require tags on all resources, restrict VM sizes, enforce encryption, deny public IP creation. Policies can audit (log non-compliance), deny (prevent creation), or remediate (auto-fix). Equivalent to AWS Config rules + SCPs combined.

Azure Blueprints - Package of policies, role assignments, ARM templates, and resource groups deployed together. Used to create compliant environments from a template. Equivalent to AWS Control Tower guardrails.

Management Groups - Hierarchy above subscriptions. Apply policies and RBAC at scale. Up to 6 levels deep. Equivalent to AWS Organization OUs.

Q15 ARM templates vs Bicep vs Terraform on Azure

The Simple Answer

ARM templates - Azure-native JSON. Verbose, hard to read. Full Azure feature support on day one. Being replaced by Bicep.

Bicep - Domain-specific language that compiles to ARM templates. Much cleaner syntax. First-class Azure support. Modules for reusability. The recommended native IaC for Azure. Equivalent to AWS CDK compiling to CloudFormation.

Terraform - Multi-cloud HCL. AzureRM provider covers most services. May lag behind Bicep on new feature support. Best for multi-cloud or teams already using Terraform.

Decision Guide

Azure-only and prefer clean syntax → Bicep. Multi-cloud → Terraform. Legacy → ARM templates (migrate to Bicep). Bicep is winning the Azure-native IaC space rapidly.

Q16 Azure Front Door + CDN - global traffic management

The Simple Answer

Azure Front Door - Global load balancer + CDN + WAF in one service. Routes traffic to the nearest healthy backend globally. SSL offloading, URL-based routing, session affinity, health probes. Equivalent to AWS CloudFront + Global Accelerator + WAF combined.

Azure CDN - Content delivery network with multiple providers (Microsoft, Akamai, Verizon). Caches static content at edge locations. Simpler than Front Door but fewer features.

When to Use Which

Front Door - Dynamic content, global applications, WAF requirement, complex routing. **CDN** - Static content delivery, simple caching. Most production applications should use Front Door.

Q17 Azure Container Apps - serverless containers, Dapr, KEDA

The Simple Answer

Container Apps is a serverless container platform built on Kubernetes (but fully managed - you never see the cluster). Scale to zero. Built-in Dapr integration for microservices patterns (service discovery, pub/sub, state management). KEDA-based autoscaling from any event source.

Container Apps vs AKS vs App Service

Container Apps - Serverless containers. No cluster management. Scale to zero. Dapr and KEDA built-in. Best for microservices that do not need full Kubernetes control.

AKS - Full Kubernetes. Maximum control. Best for complex architectures, custom operators, and specific Kubernetes features.

App Service - PaaS for web apps. Deployment slots. Best for traditional web applications without containerization.

> **Container Apps is Azure's answer to GCP Cloud Run + AWS ECS Fargate combined.** It abstracts away Kubernetes while keeping container benefits. The Dapr integration is unique - no equivalent exists in AWS or GCP's managed services.

SECTION 3 - Tricky Azure Scenarios (Q18-Q21)

Q18 AKS cluster out of IP addresses - Azure CNI exhaustion

What Happened

New Pods are stuck in Pending state. kubectl describe shows "failed to allocate for Pod: no available addresses." The AKS cluster cannot schedule more Pods because the VNet subnet has no free IP addresses.

Why It Happens

Azure CNI pre-allocates IPs for each node based on the maximum Pod count per node (default 30). A node with max 30 Pods reserves 31 IPs (30 for Pods + 1 for the node). A cluster with 10 nodes reserves 310 IPs. If the subnet is a /24 (254 usable IPs), you run out at 8 nodes.

Step-by-Step Resolution

Step 1 - Confirm IP exhaustion. Check subnet available IPs in the VNet portal page. Check node status: kubectl get nodes - nodes may be NotReady if they cannot get IPs.

Step 2 - Reduce max-pods per node. If your Pods do not actually need 30 slots per node, reduce maxPods in the node pool (e.g., 20). This releases pre-allocated IPs. Requires recreating the node pool.

Step 3 - Expand the subnet. If the VNet CIDR allows it, expand the AKS subnet. Note: you cannot resize an in-use AKS subnet in Azure CNI classic mode. You may need to create a new node pool in a different subnet.

Step 4 - Switch to Azure CNI Overlay. This mode uses a separate Pod CIDR not from the VNet. Pods get IPs from a /16 overlay network (65,000 IPs). VNet IPs are only consumed by nodes, not Pods. This is the recommended solution for new clusters and the long-term fix for existing clusters.

Step 5 - Plan subnet sizing upfront. For Azure CNI: Subnet IPs needed = (nodes × (max-pods + 1)) + reserved. For a 50-node cluster with max-pods=30: 50 × 31 = 1,550 IPs → need at least a /21 subnet.

Prevention

> Always calculate IP requirements before creating the cluster. Use Azure CNI Overlay for new clusters to avoid this problem entirely. Monitor subnet IP utilization with Azure Monitor. Alert when usage exceeds 70%.

Q19 Azure managed identity not working - RBAC debugging

What Happened

An Azure Function or App Service using a managed identity gets "403 Forbidden" when accessing a Key Vault, Storage Account, or SQL Database. The managed identity exists and is assigned to the resource, but RBAC is not working.

Step-by-Step Debugging

Step 1 - Verify the managed identity is enabled. Resource → Identity → System assigned → Status: On. Note the Object ID.

Step 2 - Verify RBAC assignment exists. Target resource (Key Vault, Storage) → Access control (IAM) → Role assignments. Search for the managed identity's Object ID. Verify the correct role is assigned (Key Vault Secrets User, Storage Blob Data Reader, etc.).

Step 3 - Check the correct role. Common mistake: assigning "Reader" (reads resource metadata) instead of "Storage Blob Data Reader" (reads blob data). "Reader" gives ARM access, not data plane access. The data-plane roles are separate.

Step 4 - Check Key Vault access model. Key Vault can use either vault access policies OR Azure RBAC. If using access policies, RBAC assignments are ignored - you must add a vault access policy instead. If using RBAC, access policies are ignored. Check: Key Vault → Settings → Access configuration.

Step 5 - Check propagation delay. RBAC assignments can take up to 10 minutes to propagate. If you just created the assignment, wait and retry.

Step 6 - Check for Deny assignments. Azure Blueprints or custom deny assignments can override role assignments. Check: Access control (IAM) → Deny assignments.

Step 7 - For AKS workload identity. Verify the Kubernetes ServiceAccount is annotated with the managed identity client ID. Verify the federated credential is configured on the managed identity for the correct namespace and ServiceAccount name.

Common Mistakes by Frequency

1. Wrong role - Reader instead of Data Reader (40%). 2. Key Vault access policy vs RBAC mismatch (25%). 3. Propagation delay - just created (15%). 4. Managed identity not enabled (10%). 5. Wrong managed identity assigned (10%).

Q20 Azure Storage account accidentally exposed publicly

Step-by-Step Response

Step 1 - Disable public access immediately. Storage Account → Networking → Public network access → Disabled. Or set to "Enabled from selected virtual networks and IP addresses" and add only your VNet.

Step 2 - Check what was exposed. Review the containers (blob) and file shares. Were there sensitive files? PII? Credentials? Configuration files with secrets?

Step 3 - Check access logs. Storage Account → Monitoring → Diagnostic settings. If logging was enabled, check for: anonymous access from unknown IPs, large data transfers (exfiltration indicators), access to sensitive containers.

Step 4 - Rotate exposed credentials. If any files contained secrets, connection strings, or API keys, rotate them immediately.

Step 5 - Assess compliance impact. If PII was exposed, determine notification requirements under GDPR, CCPA, or relevant regulations.

Step 6 - Root cause. Was it a Terraform/Bicep misconfiguration? A manual portal change? An Azure Policy that should have prevented it?

Prevention

Apply Azure Policy: "Storage accounts should disable public network access" in Deny mode. Enable Microsoft Defender for Storage (detects anomalous access patterns). Use Private Endpoints for all production storage accounts. Disable blob anonymous access at the account level (allowBlobPublicAccess: false). Review storage account networking settings quarterly.

Q21 Azure Function cold start causing SLA breach

What Happened

An HTTP-triggered Azure Function on the Consumption plan has P99 latency of 5 seconds. The SLA requires P99 under 1 second. Investigation shows cold starts adding 3-5 seconds on first invocations after idle periods.

Step-by-Step Resolution

Step 1 - Confirm cold start is the issue. Application Insights → Performance → look for bimodal latency distribution - fast responses (warm) and slow responses (cold). Filter by cold start indicator if available.

Step 2 - Understand the Consumption plan behavior. After ~20 minutes of inactivity, the Function instance is deallocated. The next request triggers a full cold start: allocate infrastructure, load the runtime, load your code, initialize dependencies.

Step 3 - Quick fix - keep warm. Add a Timer trigger function that runs every 5 minutes (does nothing except keep the instance alive). Cheap but hacky. Not guaranteed to prevent all cold starts (Azure may still deallocate).

Step 4 - Proper fix - Premium plan. Switch to Premium plan with pre-warmed instances. Always-ready instances (minimum 1) handle requests immediately. Elastic scale for traffic spikes. VNet integration included. Costs more but eliminates cold starts entirely.

Step 5 - Optimize cold start duration. Reduce deployment package size (remove unused dependencies). Use language-specific optimizations: .NET → ReadyToRun compilation, Java → GraalVM native image. Minimize initialization code - defer heavy initialization to first request, not module load.

Step 6 - Consider Azure Container Apps or App Service. If the function is an HTTP API that needs consistent low latency, Container Apps (with min replicas = 1) or App Service may be more appropriate. Functions' strength is event-driven, not always-on HTTP APIs.

Prevention

For latency-sensitive HTTP APIs, never use the Consumption plan in production. Use Premium with at least 1 pre-warmed instance. Monitor cold start metrics in Application Insights. Set alerts on P99 latency. Consider Container Apps as an alternative for HTTP-heavy workloads.

Interview Tips

For Azure-Focused Interviews

Know Azure-unique features: Managed identities (automatic credential management), Cosmos DB consistency levels, App Service deployment slots, Azure Policy for governance, Bicep for IaC, and Azure Front Door for global routing. Be able to compare with AWS equivalents.

Key Azure Advantages to Mention

Free AKS control plane. App Service deployment slots for zero-downtime deployments. Cosmos DB's five consistency levels. Azure AD integration across all services. Bicep's clean IaC syntax. Management Groups for enterprise governance. Azure DevOps as an all-in-one ALM platform.

Quick Reference Cheat Sheet

Topic	Key Takeaway
Resource hierarchy	Tenant → Management Groups → Subscriptions → Resource Groups → Resources.
Managed identities	Always use managed identities. Never store credentials. System or user-assigned.
AKS	Free control plane. Use Azure CNI Overlay to avoid IP exhaustion. Workload Identity for Azure access.
Functions	Consumption for event-driven. Premium for low latency. Durable Functions for orchestration.
App Service	Deployment slots for blue-green. Best PaaS for web apps. Swap is instant rollback.
Cosmos DB	Five consistency levels (unique). Global distribution. Multi-model API. Watch RU costs.
Key Vault	Secrets + keys + certs in one service. RBAC model preferred over access policies.
Storage	Blob tiers: Hot → Cool → Cold → Archive. Disable public access. Use Private Endpoints.
VNet	Regional (unlike GCP global). NSGs are stateful. Private Endpoints for PaaS services.
AKS IP planning	Azure CNI pre-allocates IPs. Calculate: nodes × (max-pods + 1). Use CNI Overlay for scale.
Governance	Azure Policy (deny/audit/remediate) + Management Groups + Blueprints.
IaC	Bicep (recommended native). Terraform (multi-cloud). ARM templates (legacy).
Monitoring	Azure Monitor + Log Analytics (KQL) + Application Insights. KQL is powerful.
DevOps	Azure Pipelines for CI/CD. Boards for work tracking. Artifacts for packages.
Front Door	Global LB + CDN + WAF combined. For dynamic global apps.
Container Apps	Serverless containers. Dapr built-in. Scale to zero. KEDA autoscaling. Azure-unique.

GCP Question Index - All 20 Questions

Q#	Question
Q1	GCP global infrastructure - regions, zones, projects, billing
Q2	GCP IAM - service accounts, Workload Identity Federation, org policies
Q3	Compute Engine - machine types, preemptible/spot VMs, managed instance groups
Q4	Cloud Storage - classes, lifecycle, signed URLs, IAM vs ACL
Q5	GCP VPC - Shared VPC, peering, Cloud NAT, Private Google Access, firewall rules
Q6	Cloud SQL - MySQL, PostgreSQL, SQL Server, HA, replicas
Q7	GKE - Autopilot vs Standard, Workload Identity, node pools, release channels
Q8	Cloud Run - serverless containers, scaling to zero, traffic splitting
Q9	Cloud Functions - gen2, triggers, event-driven patterns
Q10	Spanner vs Firestore vs BigQuery - database selection
Q11	Pub/Sub - topics, subscriptions, push vs pull, dead letter, ordering
Q12	Cloud Build + Artifact Registry - CI/CD on GCP
Q13	GCP networking - Cloud Load Balancing, Cloud Armor, Cloud CDN
Q14	GCP monitoring - Cloud Monitoring, Logging, Trace, Error Reporting
Q15	GCP security - Security Command Center, Binary Authorization, VPC Service Controls
Q16	GCP cost management - committed use, sustained use, recommendations
Q17	GKE Pod can't reach internet - Cloud NAT, firewall rules
Q18	GCP service account key leaked - incident response
Q19	Cloud SQL connection limits exhausted - proxy, pooling
Q20	GCS bucket data accidentally deleted - versioning recovery

SECTION 1 - Junior Core GCP (Q1-Q6)

Q1 GCP global infrastructure - regions, zones, projects, billing

The Simple Answer

GCP organizes resources in a hierarchy: Organization → Folders → Projects → Resources. Projects are the fundamental unit - every resource belongs to a project. Billing is attached to a billing account linked to one or more projects. GCP has 40+ regions, each with 3+ zones (equivalent to AWS AZs).

Key Differences from AWS

GCP projects are more like AWS accounts - they provide isolation, IAM boundaries, and billing separation. GCP Folders are like AWS OUs. GCP Organization is like AWS Organization root. The biggest difference: GCP networking is global by default - a single VPC can span all regions. AWS VPCs are regional.

Zones and Regions

Zones are independent data centers within a region. Deploy across multiple zones for HA, just like AWS AZs. Regional services (Cloud SQL HA, regional GKE clusters) automatically span zones. Multi-region services (GCS multi-region, Spanner) span multiple regions for global availability.

> **The global VPC is GCP's biggest networking advantage.** You create one VPC and subnets in any region without peering. In AWS, you need VPC peering or Transit Gateway to connect VPCs in different regions.

Q2 GCP IAM - service accounts, Workload Identity Federation, org policies

The Simple Answer

GCP IAM grants roles to members (users, groups, service accounts) on resources. Roles are collections of permissions. The hierarchy is: Organization → Folder → Project → Resource. Permissions are inherited downward - a role granted at the project level applies to all resources in that project.

Service Accounts

Service accounts are identities for applications and services (like AWS IAM roles). Each GKE Pod, Cloud Run service, or Compute Engine VM can run as a specific service account. Create dedicated service accounts per application with least-privilege roles. Never use the default Compute Engine service account in production - it has the Editor role by default, which is far too permissive.

Workload Identity Federation

Allows external identities (GitHub Actions, AWS, Azure AD) to access GCP resources without service account keys. The external identity provider is trusted by GCP, and short-lived tokens are exchanged. This is the GCP equivalent of AWS OIDC federation. Always use Workload Identity Federation over downloaded service account keys - keys are long-lived credentials that can leak.

Workload Identity (GKE-specific)

Maps Kubernetes ServiceAccounts to GCP service accounts. Pods use the Kubernetes ServiceAccount and automatically get GCP credentials - no key files, no environment variables. The recommended approach for GKE workloads accessing GCP services.

Organization Policies

Constraints applied at the organization, folder, or project level. Examples: restrict which regions resources can be created in, disable service account key creation, require uniform bucket-level access on GCS. Similar to AWS SCPs but more granular - they constrain specific resource configurations, not API actions.

Key difference from AWS IAM: GCP uses predefined roles (curated permission sets like roles/storage.objectViewer) and custom roles. AWS uses policy documents with explicit Allow/Deny on actions. GCP's approach is simpler for common use cases but less flexible for complex conditions.

Q3 Compute Engine - machine types, preemptible/spot, managed instance groups

The Simple Answer

Compute Engine provides VMs, equivalent to AWS EC2. Machine types are organized in families: general-purpose (E2, N2), compute-optimized (C2, C3), memory-optimized (M2, M3), and accelerator-optimized (A2, A3 for GPUs).

Preemptible and Spot VMs

Preemptible VMs - Up to 80% cheaper. Terminated after 24 hours max. 30-second shutdown warning. Being replaced by Spot VMs.

Spot VMs - Same pricing as preemptible but no 24-hour limit. Can run indefinitely unless GCP needs the capacity back. The current recommended option for fault-tolerant workloads.

Managed Instance Groups (MIGs)

Equivalent to AWS Auto Scaling Groups. MIGs manage a fleet of identical VMs based on an instance template. Support autoscaling (based on CPU, load balancing utilization, or custom metrics), auto-healing (replacing unhealthy VMs), rolling updates, and canary deployments.

Q4 Cloud Storage - classes, lifecycle, signed URLs

The Simple Answer

Cloud Storage (GCS) is GCP's object storage, equivalent to AWS S3. Stores objects in buckets. Virtually unlimited capacity. 99.999999999% durability (11 nines).

Storage Classes

Standard - Frequently accessed. Highest cost per GB. No retrieval fee. **Nearline** - Accessed less than once per month. Lower storage cost, retrieval fee. 30-day minimum. **Coldline** - Accessed less than once per quarter. Even lower storage, higher retrieval. 90-day minimum. **Archive** - Accessed less than once per year. Cheapest storage, highest retrieval. 365-day minimum.

Object Lifecycle Management

Rules automatically transition or delete objects based on age, creation date, or storage class. Example: move to Nearline after 30 days, Coldline after 90, Archive after 365, delete after 7 years.

Signed URLs

Temporary URLs granting time-limited access to private objects. Generated with a service account key or Workload Identity. Used for secure file downloads and uploads without making objects public.

Uniform vs Fine-Grained Access

Uniform - Bucket-level IAM only. Simpler, recommended. **Fine-grained** - Object-level ACLs (legacy). Use uniform access for new buckets. GCP org policy can enforce uniform access across the organization.

Q5 GCP VPC - Shared VPC, Cloud NAT, Private Google Access, firewall

The Simple Answer

GCP VPCs are global - subnets are regional but the VPC spans all regions. This is a fundamental difference from AWS where VPCs are regional. One GCP VPC can have subnets in us-central1, europe-west1, and asia-east1 without peering.

Shared VPC

A host project owns the VPC network. Service projects attach to it. Resources in service projects use subnets from the host project. This centralizes network management while allowing teams to manage their own compute resources. Equivalent to AWS Resource Access Manager sharing subnets, but more natively integrated.

Cloud NAT

Managed NAT gateway for private VMs to access the internet. No public IP needed on VMs. Configured per region per VPC. Automatically scales. Unlike AWS NAT Gateway which charges per GB processed, GCP Cloud NAT charges per VM using it - different cost model.

Private Google Access

Allows VMs without public IPs to reach Google APIs and services (GCS, BigQuery, etc.) through internal IP addresses. Must be enabled per subnet. Equivalent to AWS VPC Gateway Endpoints but covers all Google APIs, not just specific services.

Firewall Rules

GCP firewalls are applied at the VPC level (not subnet level like AWS NACLs). Rules can target by network tag, service account, or IP range. Default: deny all ingress, allow all egress. Firewall rules are stateful (like AWS Security Groups). Priority-based evaluation (0-65535, lower number = higher priority).

> **Key networking difference:** GCP firewall rules target instances by tags or service accounts, making them more flexible than AWS Security Groups which are attached to specific ENIs. You can apply a firewall rule to all instances with a specific tag across all regions in one rule.

Q6 Cloud SQL - HA, replicas, proxy

The Simple Answer

Cloud SQL is GCP's managed relational database service supporting MySQL, PostgreSQL, and SQL Server. Equivalent to AWS RDS.

High Availability

Regional HA creates a primary and standby instance in different zones. Automatic failover on primary failure. The standby is not readable - it is purely for HA (same as AWS RDS Multi-AZ).

Read Replicas

Cross-zone and cross-region read replicas for read scaling. Asynchronous replication with potential lag. Can be promoted to standalone instances for DR.

Cloud SQL Auth Proxy

A sidecar that handles authentication and encrypted connections to Cloud SQL. Instead of managing SSL certificates and whitelisting IPs, the proxy authenticates using the VM's or Pod's service account. Essential for GKE workloads - run as a sidecar container. Equivalent to AWS RDS Proxy but focused on authentication rather than connection pooling.

SECTION 2 - Mid-Level GCP Deep Dives (Q7-Q16)

Q7 GKE - Autopilot vs Standard, Workload Identity

The Simple Answer

GKE is Google's managed Kubernetes, widely considered the most mature managed Kubernetes offering due to Google's role as Kubernetes creator.

Autopilot vs Standard

Standard - You manage node pools (machine types, scaling, OS). Full control. Pay for the nodes whether Pods use them or not.

Autopilot - Google manages everything. You only define Pods. GCP provisions and scales nodes automatically. Pay per Pod resource request (CPU, memory). No node management, no SSH, no DaemonSets (managed by GCP). Simpler but less control.

Workload Identity

Maps Kubernetes ServiceAccounts to GCP service accounts. The recommended way for Pods to access GCP services. No service account keys mounted in Pods. Similar to AWS IRSA but simpler to configure. Enabled by default on Autopilot clusters.

Release Channels

Rapid - Latest Kubernetes version. New features first. May have bugs. **Regular** - Balanced. 2-3 months behind Rapid. Recommended for most production. **Stable** - Most tested. 4-5 months behind Rapid. For risk-averse workloads.

> **GKE vs EKS:** GKE has faster upgrades (auto-upgrade), better integrated monitoring (Cloud Monitoring with GKE dashboard), native Workload Identity, and Autopilot mode. EKS has better AWS service integration, Karpenter for node provisioning, and a larger AWS ecosystem. GKE is generally considered easier to operate.

Q8 Cloud Run - serverless containers

The Simple Answer

Cloud Run runs stateless containers without managing infrastructure. You deploy a container image, Cloud Run handles scaling (including to zero), HTTPS, and load balancing. Pay only for request processing time. Like AWS Fargate + Lambda combined - container-based but with serverless scaling.

Key Features

Scale to zero - No requests = no instances = no cost. First request triggers a cold start. **Traffic splitting** - Route percentages of traffic to different revisions for canary deployments. **Custom domains** - Map your own domain with automatic TLS. **Concurrency** - Each instance handles up to 80 concurrent requests (configurable). Unlike Lambda (one request per instance), Cloud Run maximizes instance utilization.

Cloud Run vs Cloud Functions vs GKE

Cloud Run - Containerized, scale to zero, up to 60-minute timeout, any language. Best for web APIs, microservices, and background processors.

Cloud Functions - Function-based, event-driven, 9-minute timeout (gen1) or 60-minute (gen2). Best for glue code and event handlers.

GKE - Full Kubernetes. Best for complex architectures needing fine-grained control, stateful workloads, and very high throughput.

Q9 Cloud Functions - gen2, triggers

The Simple Answer

Cloud Functions runs code in response to events without managing servers. Gen2 is built on Cloud Run (longer timeout, larger instances, concurrency, traffic splitting). Gen1 is the legacy version with more limitations.

Triggers

HTTP triggers (direct URL), Pub/Sub (message events), Cloud Storage (object events), Firestore (document changes), Eventarc (CloudEvents standard - 100+ event sources). Gen2 uses Eventarc for all non-HTTP triggers, providing a unified event model.

Q10 Spanner vs Firestore vs BigQuery - database selection

The Simple Answer

Database	Type	Best For	Scale
Cloud SQL	Relational (managed)	Traditional apps, OLTP	Vertical (up to 128 vCPUs)
Spanner	Globally distributed relational	Global OLTP, financial systems	Horizontal, global
Firestore	Document (NoSQL)	Mobile/web apps, real-time sync	Auto-scaling, serverless
Bigtable	Wide-column (NoSQL)	Time series, IoT, analytics	Petabytes, low latency
BigQuery	Analytical (data warehouse)	OLAP, business intelligence	Petabytes, serverless SQL
Memorystore	In-memory (Redis/Memcached)	Caching, sessions	Cluster mode

The Decision Framework

Need SQL with joins and transactions at moderate scale → Cloud SQL. Need SQL with global distribution and strong consistency → Spanner (expensive). Need flexible document storage for web/mobile → Firestore. Need analytics on massive datasets → BigQuery. Need caching → Memorystore.

> **Spanner is unique to GCP** — no AWS equivalent offers globally distributed SQL with strong consistency. It is GCP's differentiator for global financial and transactional workloads.

Q11 Pub/Sub - messaging

The Simple Answer

Pub/Sub is GCP's messaging service, equivalent to AWS SNS+SQS combined. Publishers send messages to topics. Subscribers receive messages from subscriptions. Supports push (HTTP delivery to endpoints) and pull (consumers poll for messages) delivery.

Key Features

At-least-once delivery - Messages may be delivered more than once. Consumers must be idempotent. **Ordering** - Optional message ordering within a key. **Dead letter topics** - Failed messages routed to a separate topic after max delivery attempts. **Exactly-once delivery** - Available with ordering enabled and Dataflow consumers. **Schema validation** - Validate message format against Avro or Protocol Buffer schemas.

Pub/Sub vs AWS SNS+SQS

Pub/Sub combines pub/sub (SNS) and queueing (SQS) in one service. Each subscription acts as an independent queue. Multiple subscriptions to one topic = fan-out (like SNS → multiple SQS). Simpler architecture - one service instead of two.

Q12 Cloud Build + Artifact Registry

The Simple Answer

Cloud Build - Serverless CI/CD. Executes build steps defined in cloudbuild.yaml. Each step runs in a container. Integrates with GitHub, GitLab, Bitbucket. Triggers on push, PR, or tag. Up to 120 minutes per build. Equivalent to AWS CodeBuild.

Artifact Registry - Multi-format package repository. Stores Docker images, npm packages, Maven artifacts, Python packages. Replaces the older Container Registry (GCR). Includes vulnerability scanning. Equivalent to AWS ECR + CodeArtifact combined.

Q13 GCP networking - Load Balancing, Cloud Armor, Cloud CDN

Cloud Load Balancing

GCP's load balancing is global by default - a single IP serves traffic from all regions, routed to the nearest healthy backend. This is fundamentally different from AWS ALB which is regional. Types: HTTP(S) Load Balancer (global L7), TCP/SSL Proxy (global L4), Network Load Balancer (regional L4), Internal Load Balancer (regional L4).

Cloud Armor

WAF and DDoS protection attached to the HTTP(S) Load Balancer. Pre-configured rules for OWASP Top 10. Custom rules for IP blocking, geo-restriction, and rate limiting. Adaptive protection uses ML to detect and mitigate L7 DDoS attacks. Equivalent to AWS WAF + Shield.

Cloud CDN

Content caching at Google's edge network. Integrated with the HTTP(S) Load Balancer. Enable with a checkbox. Cache invalidation and signed URLs supported.

Q14 GCP monitoring - Cloud Monitoring, Logging, Trace

The Simple Answer

Cloud Monitoring - Metrics, dashboards, alerting. Equivalent to CloudWatch. Automatically collects GCP service metrics. Custom metrics via OpenTelemetry or the Monitoring API. Alerting policies with notification channels (email, Slack, PagerDuty, SMS).

Cloud Logging - Centralized log management. Equivalent to CloudWatch Logs. All GCP service logs are collected automatically. Log-based metrics for alerting on log patterns. Log Router sends logs to BigQuery, GCS, Pub/Sub, or external SIEM.

Cloud Trace - Distributed tracing. Equivalent to AWS X-Ray. Visualizes request latency across microservices. Automatic instrumentation for GKE and Cloud Run. OpenTelemetry compatible.

Error Reporting - Automatically groups and tracks application errors. Shows error frequency, affected users, and stack traces. No configuration needed for supported languages.

Q15 GCP security - SCC, Binary Auth, VPC Service Controls

Security Command Center (SCC)

Centralized security dashboard. Equivalent to AWS Security Hub. Tiers: Standard (free - asset inventory, vulnerability scanning, anomaly detection) and Premium (threat detection, container security, compliance monitoring).

Binary Authorization

Ensures only trusted container images are deployed to GKE, Cloud Run, and Anthos. Requires images to be signed by specific attestors before deployment. Prevents deploying unscanned or unauthorized images. Equivalent to AWS ECR image signing + admission webhook.

VPC Service Controls

Creates security perimeters around GCP services to prevent data exfiltration. Even if an attacker compromises credentials, they cannot move data outside the perimeter. No AWS equivalent - this is a GCP-unique security feature. Used for protecting sensitive data in BigQuery, GCS, and other services.

Q16 GCP cost management

The Simple Answer

Committed Use Discounts (CUDs) - 1 or 3-year commitments for Compute Engine and Cloud SQL. Up to 57% discount. Resource-based (specific machine type) or spend-based (dollar amount). Equivalent to AWS Reserved Instances.

Sustained Use Discounts - Automatic discounts for running Compute Engine instances more than 25% of a month. No commitment needed. Up to 30% discount. GCP-unique - AWS has nothing equivalent.

Recommendations - Active Assist provides right-sizing recommendations, idle resource detection, and commitment recommendations.

> **GCP's pricing advantage:** Sustained use discounts are automatic — you get discounts without committing. Per-second billing (minimum 1 minute) for Compute Engine. Custom machine types let you specify exact vCPUs and memory, avoiding over-provisioning.

SECTION 3 - Tricky GCP Scenarios (Q17-Q20)

Q17 GKE Pod can't reach internet - Cloud NAT, firewall

Step-by-Step Debugging

Step 1 - Check if the node has a public IP. GKE private clusters have nodes without public IPs. They need Cloud NAT for outbound internet access.

Step 2 - Check Cloud NAT configuration. Is Cloud NAT configured for the VPC region and subnet? Is it applied to all instances or only specific ones? Check NAT status in the Cloud Console - it should show the subnet as covered.

Step 3 - Check firewall rules. GCP firewalls are VPC-level. Is there an egress deny rule blocking outbound traffic? The default allows all egress, but custom rules may override. Check firewall rules sorted by priority - lower number = higher priority.

Step 4 - Check if Private Google Access is needed. If the Pod needs to reach Google APIs (GCS, BigQuery) but not the public internet, enable Private Google Access on the subnet instead of Cloud NAT. It is free and does not require NAT.

Step 5 - Check DNS. Exec into the Pod: kubectl exec -it pod -- nslookup google.com. If DNS fails, check kube-dns/CoreDNS Pods. If DNS works but connections time out, it is a routing or firewall issue.

Step 6 - Check GKE network policy. If Calico NetworkPolicies are enabled, a policy might be blocking egress from the Pod's namespace.

Prevention

> Always configure Cloud NAT for GKE private clusters during cluster creation. Enable Private Google Access on all subnets. Use network tags to control which instances use NAT vs direct internet access.

Q18 GCP service account key leaked - incident response

Step-by-Step Response

Step 1 - Disable the key immediately. IAM → Service Accounts → select the SA → Keys → disable or delete the compromised key. This revokes the key within seconds.

Step 2 - Audit usage. Cloud Audit Logs → Activity logs. Filter by the service account email. Check for: unauthorized resource creation (Compute Engine instances for crypto mining), data access (GCS downloads, BigQuery exports), IAM changes (new service accounts, key creation, role grants).

Step 3 - Check all regions. Like AWS, attackers create resources in regions you do not monitor. Check Compute Engine instances across all regions.

Step 4 - Revoke active sessions. Generate new keys if the SA is still needed (prefer Workload Identity instead). Any active sessions using the old key continue until they expire - check token lifetime.

Step 5 - Clean up. Delete unauthorized resources. Remove any IAM bindings the attacker created. Review security logs for data exfiltration.

Prevention

> Eliminate service account keys entirely. Use Workload Identity for GKE, attached service accounts for Compute Engine, and Workload Identity Federation for external CI/CD. Set an organization policy to disable service account key creation (iam.disableServiceAccountKeyCreation). If keys are absolutely necessary, set key expiration and rotate automatically.

Q19 Cloud SQL connection limits exhausted

What Happened

Applications report "too many connections" errors. Cloud SQL has a per-instance connection limit (varies by tier - typically 100-4000 depending on memory).

Step-by-Step Resolution

Step 1 - Check current connections. Cloud SQL → Instance → Connections tab. Or query: SELECT COUNT(*) FROM information_schema.processlist (MySQL) or SELECT count(*) FROM pg_stat_activity (PostgreSQL).

Step 2 - Identify connection hogs. Which application or IP is holding the most connections? Are connections being properly closed after use?

Step 3 - Immediate relief. Kill idle connections from the database. Increase the instance tier (more memory = higher connection limit).

Step 4 - Implement connection pooling. Use Cloud SQL Auth Proxy as a sidecar with connection pooling. For GKE, deploy one proxy per Pod. For Cloud Run, use the built-in Cloud SQL connector. Application-side pooling (HikariCP for Java, pgBouncer for PostgreSQL) is also essential.

Step 5 - Fix the application. Ensure connections are returned to the pool after use (try-with-resources in Java, context managers in Python). Set maximum pool size per application. Configure connection timeout and idle timeout.

Prevention

> Always use connection pooling in production. Monitor active connections with Cloud Monitoring alerts. Set connection limits in the application lower than the database maximum. Use Cloud SQL Proxy for secure, pooled connections.

Q20 GCS bucket data accidentally deleted - versioning recovery

Step-by-Step Recovery

Step 1 - If versioning was enabled, objects are not truly gone - previous versions exist. List versions: gsutil ls -a gs://bucket/path/. Restore by copying the non-current version to the current: gsutil cp gs://bucket/path/file#VERSION gs://bucket/path/file.

Step 2 - If versioning was NOT enabled, the data is gone from GCS. Check for: copies in another bucket (cross-region replication), application-level backups, BigQuery exports, or any other data store that might have a copy.

Step 3 - If soft delete is enabled (new feature), deleted objects are retained for a configurable period (7-90 days) even without versioning. Check: gsutil ls --soft-deleted gs://bucket/.

Step 4 - Prevent future loss. Enable object versioning on all production buckets. Enable soft delete as an additional safety net. Set lifecycle rules to manage version costs (delete non-current versions after 90 days). Use Object Lock (Bucket Lock) for compliance data that must never be deleted. Set IAM permissions so only specific roles can delete objects - never grant storage.objects.delete to application service accounts.

Prevention

> Versioning + lifecycle rules is the minimum for any production GCS bucket. The cost of storing a few extra versions is negligible compared to the cost of data loss. Soft delete provides a safety net even for unversioned buckets.

Interview Tips

For GCP-Focused Interviews

Know the GCP-unique features: global VPC, Spanner, VPC Service Controls, sustained use discounts, BigQuery, and Cloud Run's concurrency model. Be able to compare with AWS equivalents - interviewers often ask "how would you do this in GCP vs AWS?"

Key GCP Advantages to Mention

GKE is the most mature managed Kubernetes. Global load balancing with a single IP. BigQuery for serverless analytics. Spanner for globally consistent SQL. Sustained use discounts without commitment. Cloud Run's scale-to-zero with container concurrency.

Quick Reference Cheat Sheet

Topic	Key Takeaway
Global VPC	GCP VPCs span all regions. AWS VPCs are regional. Major advantage.
IAM	Use Workload Identity (GKE) and WIF (external). Never download service account keys.
GKE	Autopilot for simplicity. Standard for control. Workload Identity for GCP access.
Cloud Run	Serverless containers. Scale to zero. Concurrency per instance (unlike Lambda).
Spanner	Globally distributed SQL. GCP-unique. For financial/global transactional workloads.
BigQuery	Serverless analytics. Petabyte scale. SQL interface. GCP's killer service.
Cloud NAT	Required for private GKE clusters. Configure during cluster creation.
Private Google Access	Free. VMs without public IPs reach Google APIs. Enable on all subnets.
Firewall rules	VPC-level, tag-based targeting. Priority order (lower = higher).
Cost	Sustained use discounts are automatic. Custom machine types avoid over-provisioning.
Storage	Standard → Nearline → Coldline → Archive. Enable versioning + lifecycle.
VPC Service Controls	GCP-unique. Prevents data exfiltration. No AWS equivalent.
Binary Auth	Only signed images deploy. Use with Artifact Registry scanning.